DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 10
Testes com banco real

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Na semana passada a API passou a separar quem pode do que não pode. Como saber que essas regras continuam valendo depois da próxima mudança, sua ou da IA? Esta apostila apresenta os fundamentos de teste de software (níveis, técnicas de projeto de casos, dublês, TDD) e mostra como o ServiceHub prova suas regras em três camadas: unitários com JUnit 5 e Mockito, fatia web com MockMvc e integração contra o mesmo PostgreSQL da produção, com Testcontainers.

Conteúdo

1. Objetivos da semana
2. Por que testar
3. Caixa-preta, caixa-branca e níveis
4. Projetando casos de teste
5. Dublês de teste
6. TDD e testes limpos
7. A pirâmide de testes
8. Ferramentas: JUnit, AssertJ, Mockito
9. Banco real com Testcontainers
10. Concorrência e cobertura
11. Aplicação no ServiceHub
12. Construindo com IA
13. Glossário
14. Exercícios
15. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. distinguir verificação de validação, caixa-preta de caixa-branca e os níveis de teste;
2. projetar casos com partição de equivalência e valor-limite, e escolher o dublê certo para cada dependência;
3. aplicar o ciclo do TDD e escrever testes legíveis (nome `deveXQuandoY`, padrão AAA, F.I.R.S.T.);
4. escrever testes unitários com JUnit 5, AssertJ e Mockito, fatias web com `@WebMvcTest` e testes de integração com Testcontainers;
5. provar uma regra sob concorrência e ler um relatório de cobertura sem confundir cobertura com qualidade.

2. Por que testar

↑ conteúdo

Testar é executar o software com a intenção de encontrar defeitos antes que o usuário os encontre. A motivação é econômica: quanto mais tarde um defeito aparece, mais caro ele fica. No editor, custa uma linha; no code review, uma conversa; em produção, pode custar dados corrompidos, clientes perdidos e uma correção feita sob pressão.

> **PARA LEMBRAR**
>
> Teste não prova ausência de defeitos. Ele reduz o risco e mostra que, *nos casos escolhidos*, o comportamento é o esperado. Por isso escolher bem os casos (seção 4) importa tanto quanto escrevê-los.

Verificação e validação

Verificação

Estamos construindo o produto corretamente?
Confere o software contra a especificação. Exemplo: a regra diz "nota de 1 a 5"; o teste envia 6 e espera 400.

Validação

Estamos construindo o produto correto?
Confere o software contra a necessidade real de quem usa. Exemplo: o cliente precisava avaliar também o prestador, e a especificação esqueceu.

Testes automatizados fazem verificação: comparam o código com o que está escrito. Validação pede gente: demonstração, teste de aceitação, conversa com o usuário. Um software pode passar em todos os testes e ainda ser o produto errado. No ServiceHub, o Mapa de Construção é a especificação que os testes verificam; a banca do fim do semestre é uma forma de validação.

3. Caixa-preta, caixa-branca e níveis

↑ conteúdo

Critério	Caixa-preta (funcional)	Caixa-branca (estrutural)
O que se conhece	Só entradas, saídas e a especificação	O código: caminhos, condições, laços
Pergunta	A resposta está certa para esta entrada?	Todo caminho do código foi exercitado?
Técnicas	Partição de equivalência, valor-limite, tabela de decisão	Cobertura de comandos, de ramos e de caminhos
Acha	Requisito mal implementado	Código que nenhum teste tocou
No ServiceHub	Integração via HTTP: status e corpo	Unitário do service e relatório do JaCoCo

Um teste que simula o repositório com Mockito fica no meio: conhece a estrutura interna (quais dependências o service usa), mas confere o resultado de fora. Há quem chame isso de caixa-cinza. A bancada de testes que o professor usa na banca é caixa-preta pura: conhece só o contrato do Mapa, por isso serve para a API de qualquer equipe.

Níveis de teste

Nível	O que testa	No ServiceHub
Unidade	A menor parte isolada: um método, uma classe	`RequestStatusTest`, services com Mockito
Integração	Partes trabalhando juntas: camadas, banco, segurança	`@SpringBootTest` com PostgreSQL real
Sistema	O sistema inteiro contra os requisitos, como caixa-preta	Bancada de testes e Postman contra a API no ar
Aceitação	Se o usuário ou cliente aceita o que foi entregue	Regras RN do Mapa conferidas na banca

Regressão não é um nível a mais: é reexecutar os outros níveis sempre que o código muda, para ver se o que funcionava continua funcionando. A automação existe principalmente para tornar a regressão barata. Rodar duzentos testes a cada commit só é viável se ninguém precisar clicar.

E O @WEBMVCTEST?

Fica entre unidade e integração. Ele sobe só a camada web (controller, validação, conversão JSON, tratamento de erros) e substitui o service por um dublê. É uma *fatia* da aplicação, não a aplicação inteira.

4. Projetando casos de teste

↑ conteúdo

As entradas possíveis de qualquer método útil são infinitas. As duas técnicas a seguir, ambas de caixa-preta, escolhem poucos casos que representam bem o todo. Elas nascem da especificação, não do código.

Partição de equivalência

Divida as entradas em **classes** que o sistema deve tratar do mesmo jeito. Se um valor de uma classe funciona, os outros da mesma classe devem funcionar; basta testar um representante de cada classe, válida ou inválida.

Exemplo genérico: um cadastro aceita idade de 18 a 120 anos.

Classe	Regra	Representante
Inválida, abaixo	Menor que 18	10
Válida	De 18 a 120	40
Inválida, acima	Maior que 120	150
Inválida, tipo	Vazio ou texto	"abc"

Quatro testes cobrem infinitas entradas. Sem a técnica, ou se testa demais o mesmo caso, ou se esquece uma classe inteira. No ServiceHub, as classes do e-mail de cadastro são: válido, sem arroba, vazio e *já existente*. Esta última é a que código gerado por IA mais esquece, e no contrato ela vira 409 (RN01).

Análise de valor-limite

Os defeitos se concentram nas bordas das classes: um `<` no lugar de `<=`, um intervalo começando em 0 em vez de 1. Por isso testa-se cada limite e o vizinho imediato de cada lado. Para idade de 18 a 120: **17, 18, 120 e 121**.

No ServiceHub, a nota da avaliação é limitada no DTO com `@Min(value = 1)` e `@Max(value = 5)`. Os valores-limite são 0, 1, 5 e 6, e quatro linhas de um teste parametrizado provam as duas bordas:

```
@ParameterizedTest
@CsvSource({"0, 400", "1, 201", "5, 201", "6, 400"})
void deveValidarLimitesDaNota(int nota, int statusEsperado) { ... }
```

Partição escolhe as classes; valor-limite escolhe os valores dentro delas. O teste acima é um exercício proposto: para o 201, o Arrange precisa de uma ordem COMPLETED.

5. Dublês de teste

↑ conteúdo

Dublê de teste é tudo que substitui uma dependência real durante o teste, como o dublê de cena substitui o ator. Serve para isolar a unidade, deixar o teste rápido e montar cenários difíceis de produzir com a dependência real ("o banco não achou o id"). A nomenclatura abaixo vem de Gerard Meszaros e é usada também por Robert C. Martin.

Tipo	O que é	Quando usar
Dummy	Objeto passado só para preencher um parâmetro; nunca é usado	O método exige o argumento, mas o caso não depende dele
Stub	Devolve respostas prontas às chamadas	Montar um cenário: "o repositório não acha o id"
Mock	Registra as chamadas para você conferir depois	A chamada é o resultado: "não deve gravar nada"
Spy	Envolve um objeto real e registra o que passou por ele	Manter o comportamento real e espiar uma chamada
Fake	Implementação simplificada que funciona de verdade	Repositório em memória, servidor de e-mail falso

Exemplo genérico: para testar um serviço que envia o boleto por e-mail, um *stub* do gateway de pagamento devolve "pago"; um *mock* do serviço de e-mail permite conferir que exatamente um e-mail saiu; um *fake* seria um servidor SMTP em memória que guarda as mensagens.

CONFUSÃO COMUM

No dia a dia todo mundo diz "mock" para tudo. O que importa é saber se você está *configurando respostas* (stub) ou *conferindo chamadas* (mock). Conferir chamada demais acopla o teste à implementação: qualquer refatoração quebra o teste sem que o comportamento tenha mudado.

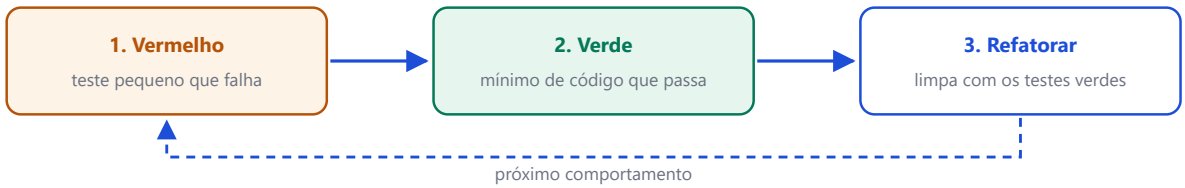
Repare que o **H2**, banco em memória muito usado em tutoriais, é um *fake* do PostgreSQL. Parece, mas não é. Bom para lógica, ruim para provar SQL e migrações. A seção 9 volta a isso.

6. TDD e testes limpos

↑ conteúdo

O ciclo vermelho, verde, refatorar

TDD (*Test-Driven Development*, desenvolvimento guiado por testes) inverte a ordem habitual: o teste vem antes do código.



O ciclo curto do TDD: cada volta acrescenta um comportamento pequeno.

Robert C. Martin resume a disciplina em **três leis**: (1) não escreva código de produção sem antes ter um teste que falha; (2) não escreva mais teste do que o suficiente para falhar, e não compilar também é falhar; (3) não escreva mais código de produção do que o suficiente para o teste passar.

O maior ganho do TDD é ter certeza de que o teste *consegue* falhar. Um teste que nunca foi visto vermelho pode estar testando nada: um assert esquecido, um caminho que nunca executa. Com IA isso fica ainda mais útil: você escreve o teste vermelho, que é a especificação executável, e pede o código que o deixa verde.

Nomes deveXQuandoY e o padrão AAA

O nome do teste diz o resultado esperado e a condição, como em `deveRetornar403QuandoEditarPerfilDeOutro`. Quando falha, o relatório já conta o que quebrou. O corpo segue **AAA**: *Arrange* (prepara o cenário), *Act* (executa uma ação), *Assert* (confere). Uma ação por teste. Exemplo genérico:

```
@Test
void deveAplicarDescontoDe10QuandoClienteTem60AnosOuMais() {
    var cliente = new Cliente(60);                 // Arrange
    int desconto = calculadora.desconto(cliente);  // Act
    assertThat(desconto).isEqualTo(10);            // Assert
}
```

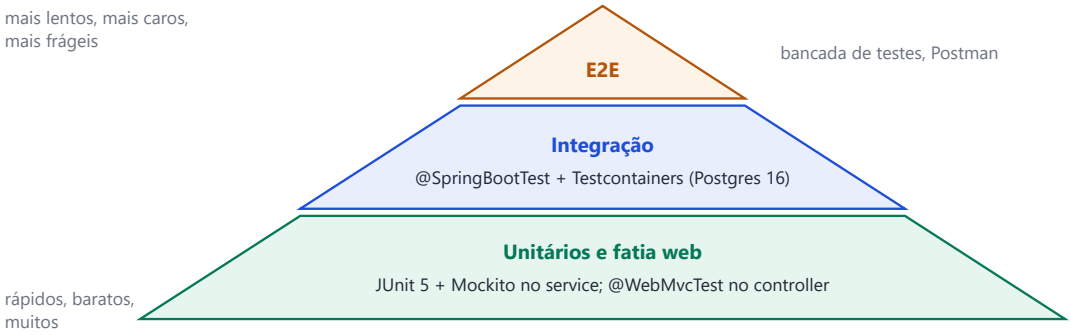
F.I.R.S.T.

Letra	Significa	Quebra quando
Fast (rápido)	Roda em segundos, então você roda sempre	Cada teste sobe um contexto novo
Independent	Nenhum teste depende de outro nem da ordem	Um teste usa dado criado por outro
Repeatable	Mesmo resultado em qualquer máquina e ambiente	Depende de rede, relógio ou banco local
Self-validating	Passa ou falha sozinho, sem ler log	Só imprime no console
Timely (oportuno)	Escrito junto com o código, ou antes	Fica para "depois da entrega"

Código de teste é código de produção: legível, sem duplicação, um conceito por teste. Teste sujo é abandonado, e com ele vai a segurança para mudar o código. Um `Thread.sleep` no teste viola o F e, muitas vezes, o R.

7. A pirâmide de testes

↑ conteúdo



Quanto mais alto na pirâmide, mais o teste cobre de uma vez e mais caro ele é de rodar e manter.

A forma da pirâmide é sobre custo e velocidade: a base tem muitos testes rápidos; o topo, poucos testes lentos que exercitam o sistema inteiro pela rede. O desenho é guia, não lei. No ServiceHub a camada de integração é grande de propósito, porque as regras de segurança e de banco (401, 403, travas, migrações) só se provam com tudo ligado.

8. Ferramentas: JUnit, AssertJ, Mockito

↑ conteúdo

Tudo vem no `spring-boot-starter-test`: o **JUnit 5** executa os testes, o **AssertJ** confere os resultados e o **Mockito** cria os dublês.

JUnit 5

Anotação	Para quê
`@Test`	Marca um caso de teste
`@DisplayName`	Nome legível no relatório, por exemplo citando a regra RN do Mapa
`@BeforeEach`	Prepara o cenário antes de cada teste
`@ParameterizedTest` + `@CsvSource`	Um teste, várias linhas de dados
`@RepeatedTest(5)`	Repete o teste, útil para pegar falha intermitente
`@ExtendWith(MockitoExtension.class)`	Liga `@Mock` e `@InjectMocks` num teste sem Spring

`@Disabled` existe, mas teste desligado sem motivo escrito é dívida escondida. Para rodar uma classe só: `mvnw.cmd test -Dtest=RequestStatusTest`.

AssertJ: asserções que se leem como frase

JUnit nativo

```
assertEquals(HttpStatus.FORBIDDEN,
    resp.getStatusCode());
assertThrows(NotFound.class,
    () -> service.findById(42L));
```

AssertJ

```
assertThat(resp.getStatusCode())
    .isEqualTo(HttpStatus.FORBIDDEN);
assertThatThrownBy(
        () -> service.findById(42L))
    .isInstanceOf(NotFound.class)
    .hasMessageContaining("42");
```

A vantagem aparece quando o teste falha: a mensagem mostra o objeto inteiro, a coleção esperada e a recebida. Asserções úteis em API: `containsEntry("type", "urn:servicehub:problem:forbidden")` para o corpo de erro e `containsExactlyInAnyOrder(...)` para coleções sem ordem garantida.

Mockito

Num teste com `@ExtendWith(MockitoExtension.class)`, `@Mock` cria um dublê que devolve vazio até ser configurado, `when(...).thenReturn(...)` faz dele um stub e `verify(...)` faz dele um mock. `@InjectMocks` cria a classe real sob teste com os dublês injetados no construtor. A regra de ouro: mocke as dependências, nunca a classe que você está testando.

@WebMvcTest e @MockitoBean

Para testar o controller sem banco, `@WebMvcTest(MeuController.class)` sobe só a camada web e o `MockMvc`, que simula requisições HTTP sem servidor de verdade. O service vira um dublê registrado no contexto do Spring com `@MockitoBean`. O que se prova ali: rota, validação, status code, JSON de erro e header `Location`.

DIFERENÇAS DO SPRING BOOT 4

Tutoriais e IA treinados em Boot 2 e 3 trazem os nomes antigos. No Boot 4:

- `@MockBean` foi removido; use `@MockitoBean`, de `org.springframework.test.context.bean.override.mockito`;
- `@WebMvcTest` e `@AutoConfigureMockMvc` vêm de `org.springframework.boot.webmvc.test.autoconfigure`.

No ServiceHub as fatias web usam `@AutoConfigureMockMvc(addFilters = false)`, que desliga os filtros de segurança. A decisão é consciente: a fatia prova o contrato HTTP do controller; a segurança se prova na integração, com token de verdade.

9. Banco real com Testcontainers

↑ conteúdo

Um teste de integração com outro banco prova que a API funciona *em outro banco*. Muitos tutoriais ensinam H2 em memória com `@AutoConfigureTestDatabase`, que troca o datasource por um banco embutido. O ServiceHub não faz isso (ADR-0005).

Critério	H2 em memória	Testcontainers (PostgreSQL 16)
Migrações	`BIGSERIAL`, `CHECK` e `ALTER` das migrações quebram ou mudam de sentido	As mesmas da produção, aplicadas pelo Flyway
Schema	Tentação de usar `ddl-auto=create-drop`	`ddl-auto=validate`, como na produção
Travas e SQL	`FOR UPDATE` e dialeto diferentes	Comportamento real do Postgres
Custo	Nenhum pré-requisito	Docker rodando; alguns segundos por JVM

O ganho é a paridade: um erro de SQL ou de migração aparece no teste, não no deploy. O custo é ter Docker rodando, o mesmo Docker das Semanas 11 e 12.

Como o Testcontainers funciona

O **Testcontainers** é uma biblioteca que, durante o teste, pede ao Docker um container descartável (aqui, `postgres:16-alpine`), espera ele ficar pronto e o destrói no fim. Não precisa de Compose: a biblioteca fala direto com o Docker. No Spring Boot, a anotação `@ServiceConnection` sobre o bean do container entrega URL, usuário e senha ao datasource, sem nenhuma propriedade escrita à mão.

> **TESTCONTAINERS 2.X**
>
> Os módulos mudaram de nome: `testcontainers-postgresql` e `testcontainers-junit-jupiter`, e a classe agora é `org.testcontainers.postgresql.PostgreSQLContainer`. Código antigo traz `org.testcontainers.containers.PostgreSQLContainer`, que não existe mais nesse pacote.

Uma classe base com banco limpo

Todos os testes de integração herdam de uma classe base que sobe a aplicação inteira numa porta aleatória (`RANDOM_PORT`), importa a configuração do container e, antes de cada teste, apaga os dados com `TRUNCATE ... RESTART IDENTITY CASCADE`. É assim que se garante o I e o R do F.I.R.S.T.: nenhum teste depende da ordem nem do lixo de outro, e os ids recomeçam do 1. Como todas as classes herdam a mesma configuração, o Spring reaproveita o contexto (e o container) entre elas, o que mantém a suíte razoavelmente rápida.

10. Concorrência e cobertura

↑ conteúdo

Por que concorrência é difícil de testar

Uma **condição de corrida** (*race condition*) acontece quando o resultado depende da ordem em que as threads executam. A forma mais comum é o padrão *verificar e depois agir* (check-then-act): a verificação fica velha antes da ação.

```
Thread A: lê status = ACCEPTED
Thread B: lê status = ACCEPTED
Thread A: grava IN_PROGRESS   ✓
Thread B: grava CANCELLED     ✓   // as duas "venceram"; a última apagou a primeira
```

Testar isso é difícil por dois motivos. O sistema operacional decide a intercalação das threads, então o mesmo teste passa muitas vezes seguidas e de repente falha (não determinismo). E um log ou breakpoint muda o tempo das threads e esconde a falha. A estratégia é: forçar as threads a começar juntas, repetir o teste e conferir uma **invariante** (exatamente uma vence), não a ordem.

Em Java, a ferramenta para "começar juntas" é a `CyclicBarrier`: cada thread chama `await()` e fica parada até que todas as participantes cheguem; aí todas são liberadas ao mesmo tempo. O antídoto da condição de corrida, no banco, é a **trava pessimista**: `SELECT ... FOR UPDATE` trava a linha até o fim da transação, e a thread B espera e relê o estado já alterado. É mais um motivo para testar no Postgres real.

Cobertura: o que mede e o que não mede

Cobertura é a fração do código executada pelos testes, medida em linhas, ramos (cada lado de um `if`) ou métodos. É uma técnica de caixa-branca. Exemplo genérico:

```
int desconto(int idade) {
    int d = 0;
    if (idade >= 60) d = 10;
    return d;
}
// um único teste: desconto(70) == 10
// 100% das linhas executadas, mas só 50% dos ramos: idade < 60 nunca rodou
```

Mede

Quais linhas, ramos e métodos foram executados por algum teste.

Não mede

Se o teste conferiu o resultado; se o requisito que falta no código foi testado; se as combinações de entrada foram cobertas.

Cobertura baixa prova que falta teste. Cobertura alta não prova que os testes são bons: um teste sem `assertThat` cobre a linha e não prova nada. E código que nunca foi escrito, como uma

validação esquecida, não aparece em relatório nenhum; só a especificação revela essa falta. O número é sintoma, não meta.

O **JaCoCo** é a ferramenta de cobertura do ecossistema Java. Ele instrumenta o bytecode durante os testes e gera um relatório HTML com as linhas em verde (executada), amarelo (só parte dos ramos) e vermelho (nunca executada). Leia o vermelho primeiro, começando pelos services: linha nunca executada é regra nunca provada.

11. Aplicação no ServiceHub

↑ conteúdo

Todos os trechos abaixo são testes reais de `src/test`, alguns resumidos.

Domínio puro: a máquina de estados em uma tabela

`model/RequestStatusTest.java` (resumido) testa as 25 combinações de origem e destino da ordem (RN08), sem Spring e sem mock:

```
@ParameterizedTest(name = "{0} -> {1}: transição válida={2}, prestador={3}, cliente={4}")
@CsvSource({
    "PENDING,     ACCEPTED,    true,  true,  false",
    "PENDING,     CANCELLED,   true,  true,  true",
    "COMPLETED,   CANCELLED,   false, false, false",
    // ... 25 linhas: 5 x 5 combinações
})
```

Testar todas as combinações pega também o que *não* deveria ser permitido. Só cinco transições existem; as outras vinte são proibidas, e é nelas que mora o bug.

Service com Mockito: stub e mock no mesmo teste

Em `service/UserServiceTest.java` (com comentários acrescentados):

```
@Test
void deveLancarEmailAlreadyExistsQuandoEmailJaCadastrado() {
    when(repository.existsByEmail("maria@x.com")).thenReturn(true);   // stub
    assertThatThrownBy(() -> service.register(dto("maria@x.com")))
            .isInstanceOf(EmailAlreadyExistsException.class);
    verify(repository, never()).saveAndFlush(any());                  // mock
}
```

O `verify(..., never())` aqui é legítimo: não gravar é o próprio comportamento exigido pela RN01.

Fatia web

Cabeçalho de `controller/ServiceOfferingControllerTest.java` (resumido):

```
@WebMvcTest(ServiceOfferingController.class)
@AutoConfigureMockMvc(addFilters = false)
class ServiceOfferingControllerTest {
    @Autowired MockMvc mockMvc;
    @MockitoBean ServiceOfferingService service;
    ...
}
```

É nesta fatia que o `JwtUserConverter` quebraria se fosse `@Component` (Semana 09): o `@WebMvcTest` carrega todo `Converter` e não há repositório no contexto.

O container e a classe base

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {
    @Bean
    @ServiceConnection
    PostgreSQLContainer postgresContainer() {
        return new PostgreSQLContainer("postgres:16-alpine");
    }
}
```

`AbstractIntegrationTest.java` (resumido; omite outras properties, como as do admin inicial):

```
@SpringBootTest(webEnvironment = RANDOM_PORT,
        properties = "servicehub.jwt.secret=" + AbstractIntegrationTest.JWT_SECRET)
@AutoConfigureTestRestTemplate
@Import(TestcontainersConfiguration.class)
public abstract class AbstractIntegrationTest {
    @Autowired protected TestRestTemplate rest;
    @Autowired protected JdbcTemplate jdbc;

    @BeforeEach
    void limparBanco() throws Exception {
        jdbc.execute("TRUNCATE TABLE reviews, service_requests, services, users"
                + " RESTART IDENTITY CASCADE");
        // recria o ADMIN inicial apagado pelo TRUNCATE
    }
}
```

No Boot 4 o `TestRestTemplate` mudou para o módulo `spring-boot-resttestclient` e precisa de `@AutoConfigureTestRestTemplate`. A classe também oferece atalhos (`registrar`, `login`, `adminToken`, `get`, `post`, `put`, `patch`, `delete`) que deixam cada teste curto.

A Semana 09 provada de ponta a ponta

Em `integration/UserSecurityIntegrationTest.java` (resumido), um teste prova três regras com banco, segurança e HTTP reais:

```
@Test
@DisplayName("RN11: exclusão pelo ADMIN é lógica, bloqueia login e desativa"
        + " os serviços do usuário")
void deveDesativarUsuarioEServicosQuandoAdminExclui() {
    long servicoId = criarServico(joao, "Instalação de tomadas", "150.00", "ELETRICA");

    assertThat(delete("/api/v1/users/" + joaoId, maria).getStatusCode())
            .isEqualTo(HttpStatus.FORBIDDEN);
    assertThat(delete("/api/v1/users/" + joaoId, adminToken()).getStatusCode())
            .isEqualTo(HttpStatus.NO_CONTENT);

    assertThat(get("/api/v1/auth/me", joao).getStatusCode())
            .isEqualTo(HttpStatus.UNAUTHORIZED);
    assertThat(get("/api/v1/services/" + servicoId, null).getStatusCode())
            .isEqualTo(HttpStatus.NOT_FOUND);
}
```

O token antigo de João passa a dar 401: é o converter funcionando. Além disso, `ProtectedRoutesIntegrationTest` varre todas as rotas protegidas sem token e exige 401; rota nova entra no CSV dele.

Concorrência

Em `integration/ReviewFixesIntegrationTest.java` (resumido), duas mudanças de status chegam juntas numa ordem ACCEPTED:

```
@RepeatedTest(5)
@DisplayName("RN08: mudanças de status simultâneas não furam a máquina de estados"
        + " (um 200, um 409)")
void deveSerializarMudancasDeStatusConcorrentes() throws Exception {
    ...
    List<HttpStatusCode> status = concurrently(
        () -> patch(url, joao, Map.of("status", "IN_PROGRESS")).getStatusCode(),
        () -> patch(url, maria, Map.of("status", "CANCELLED")).getStatusCode());
    assertThat(status).containsExactlyInAnyOrder(HttpStatus.OK, HttpStatus.CONFLICT);
}

private static List<HttpStatusCode> concurrently(Supplier<HttpStatusCode> a,
                                                 Supplier<HttpStatusCode> b) {
    CyclicBarrier barrier = new CyclicBarrier(2);
    Supplier<HttpStatusCode> wa = () -> { await(barrier); return a.get(); };
    Supplier<HttpStatusCode> wb = () -> { await(barrier); return b.get(); };
    CompletableFuture<HttpStatusCode> fa = CompletableFuture.supplyAsync(wa);
    CompletableFuture<HttpStatusCode> fb = CompletableFuture.supplyAsync(wb);
    return List.of(fa.join(), fb.join());
}
```

O que garante "um 200 e um 409" é a trava no repositório, que vira `SELECT ... FOR UPDATE`:

```
/** Trava a linha da ordem até o fim da transação: duas mudanças de status
    simultâneas viram fila (RN08). */
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT r FROM ServiceRequest r WHERE r.id = :id")
Optional<ServiceRequest> findByIdForUpdate(@Param("id") Long id);
```

Cobertura no build

O `pom.xml` declara o `jacoco-maven-plugin`; `mvnw.cmd verify` roda tudo e gera `target/site/jacoco/index.html`. Um detalhe que costuma quebrar o relatório: o Surefire carrega o Mockito como agente e o `argLine` precisa começar com `@{argLine}`, senão o agente do JaCoCo some e o relatório sai vazio:

```
<argLine>@{argLine} -javaagent:${org.mockito:mockito-core:jar} -Xshare:off</argLine>
```

12. Construindo com IA

↑ conteúdo

Pedir código e testes juntos gera testes que confirmam o código, não a regra: se um está errado, o outro erra junto. Por isso esta semana inverte a ordem.

Movimento	O que você faz nesta semana
Especificar	Escolhe uma regra RN do Mapa e escreve os nomes dos testes: `deveRetornar409QuandoAvaliarOrdemDuasVezes`.
Gerar	Pede primeiro só o teste, sem tocar na produção; roda e vê vermelho. Depois pede a implementação mínima.
Revisar	Assert do status *e* do `type`? Postgres real, não H2? O nome bate com o que o corpo prova?
Provar	Remove a regra de propósito, vê o teste falhar, restaura e vê verde.

Prompt fraco e prompt bom

Fraco

```
escreve testes pro meu ReviewService
```

Vêm testes que espelham o código atual, com `@MockBean` e H2, e asserts só de status 200.

Bom

Nomeia a regra, os casos e os status, fixa as versões, proíbe mexer na produção na primeira etapa e exige ver o vermelho antes do verde.

O prompt desta semana

A entrega é a base de testes com banco real e uma regra do Mapa construída em TDD. O exemplo usa a RN09; troque pela regra que a equipe escolher. O prompt parte da API da Semana 09 (JWT, papéis e regras de dono já prontos):

```
Contexto: API ServiceHub em Java 21, Spring Boot 4.1, Spring Security 7
(JWT), PostgreSQL 16 + Flyway (ddl-auto=validate). Testes com JUnit 5,
AssertJ, Mockito e Testcontainers 2 (org.testcontainers.postgresql).
Docker está rodando.

Objetivo: construir em TDD a regra RN09 do Mapa: "Só quem abriu a ordem
avalia, só depois de COMPLETED, uma única vez" (403 / 409), na rota
POST /api/v1/service-requests/{id}/reviews.

Etapa 1, SÓ TESTES (não altere nada em src/main):
- Se ainda não existirem, crie TestcontainersConfiguration (bean
  PostgreSQLContainer("postgres:16-alpine") com @ServiceConnection) e
```

```
  uma classe base @SpringBootTest(RANDOM_PORT) com
  @AutoConfigureTestRestTemplate, @Import da configuração e TRUNCATE
  ... RESTART IDENTITY CASCADE no @BeforeEach.
- Testes de integração, um por caso, nome deveXQuandoY, padrão AAA,
  @DisplayName começando por "RN09:":
  deveRetornar201QuandoClienteAvaliaOrdemConcluida
  deveRetornar403QuandoNaoForClienteDaOrdem
  deveRetornar409QuandoOrdemNaoConcluida
  deveRetornar409QuandoAvaliarOrdemDuasVezes
- Cada erro confere status E o "type" do ProblemDetail.
- Tokens obtidos por login real; nada de desligar a segurança.
- NÃO use H2, @AutoConfigureTestDatabase nem @MockBean.
Rode mvnw.cmd verify e me mostre quais testes falham e por quê.
Pare e espere.

Etapa 2 (só depois que eu confirmar o vermelho): implemente o mínimo
no service para os testes passarem, sem alterar os testes.

Antes de cada etapa, mostre o plano (arquivos e casos).

Critério de aceite: mvnw.cmd verify verde, e cada teste falha se a
checagem correspondente da RN09 for removida do service.
```

O que a IA costuma errar nesta etapa

- `@MockBean` (removido no Boot 4) em vez de `@MockitoBean`; imports de Testcontainers 1.x e de Boot 3 para `TestRestTemplate` e `@WebMvcTest`.
- H2 ou `@AutoConfigureTestDatabase`, que trocam o banco e quebram as migrações Postgres.
- Teste que só confere status 200, ignora o corpo, ou não tem assert nenhum.
- Mockar a própria classe sob teste, ou ajustar o código (ou o teste) até ficar verde.
- Testes que dependem da ordem de execução ou de dados criados por outro teste.

Checklist de revisão

☐ Os testes de integração herdam da classe base e rodam contra Postgres em container?

☐ Cada caso de erro confere status e `type` do ProblemDetail?

☐ Os nomes seguem `deveXQuandoY` e o `@DisplayName` cita a RN?

☐ Você viu cada teste vermelho antes de vê-lo verde?

☐ O relatório do JaCoCo mostra as linhas da regra em verde?

Prova

`mvnw.cmd verify` verde com o Docker ligado. Comente no service a checagem "já avaliada": `deveRetornar409QuandoAvaliarOrdemDuasVezes` fica vermelho. Restaure e veja o verde; registre no PR os dois commits ou um print. É o ritual da banca.

13. Glossário

↑ conteúdo

Verificação	Conferir o software contra a especificação: o produto está sendo construído corretamente?
Validação	Conferir o software contra a necessidade real: é o produto correto?
Caixa-preta	Teste que conhece só entradas, saídas e a especificação.
Caixa-branca	Teste guiado pela estrutura interna do código.
Regressão	Reexecutar os testes após uma mudança para ver se o que funcionava continua funcionando.
Partição de equivalência	Dividir as entradas em classes tratadas do mesmo jeito e testar um representante de cada.
Valor-limite	Testar as bordas de cada classe e os vizinhos imediatos.
Dublê de teste	Substituto de uma dependência real: dummy, stub, mock, spy ou fake.
Stub / mock	Dublê que devolve respostas prontas / que registra chamadas para conferência.
TDD	Desenvolvimento guiado por testes: vermelho, verde, refatorar.
AAA	Arrange, Act, Assert: estrutura de um teste.
F.I.R.S.T.	Fast, Independent, Repeatable, Self-validating, Timely.
Fatia (slice)	Teste que sobe só uma camada do Spring, como `@WebMvcTest`.
Testcontainers	Biblioteca que sobe containers descartáveis, como um Postgres, durante os testes.
Condição de corrida	Resultado que depende da ordem de execução de threads concorrentes.
Cobertura	Fração de linhas, ramos ou métodos executados pelos testes.

14. Exercícios

↑ conteúdo

1. Dê um exemplo, no ServiceHub, de um defeito que os testes automatizados pegariam (verificação) e de outro que só uma conversa com o usuário revelaria (validação).
2. Classifique cada teste em unidade, integração ou sistema, e em caixa-preta ou caixa-branca: (a) `RequestStatusTest`; (b) a bancada de testes do professor rodando o ciclo completo; (c) `UserSecurityIntegrationTest`; (d) `UserServiceTest` com Mockito.
3. No DTO do ServiceHub, o preço de um serviço vai de 0,01 a 99.999.999,99, com no máximo duas casas decimais. Liste as classes de equivalência e os valores-limite que você testaria.
4. Para cada situação, diga o tipo de dublê: (a) o repositório deve responder "e-mail já existe"; (b) conferir que nada foi gravado; (c) um repositório em memória com `HashMap`; (d) um parâmetro obrigatório que o caso não usa.

5. Explique por que um teste que nunca foi visto falhar não prova nada. Como o TDD resolve isso?
6. Reescreva o nome e organize em AAA: `@Test void teste1() { assertEquals(404, client.get("/services/999").status()); }`.
7. Um teste cria um usuário com e-mail `ana@x.com` e outro teste, da mesma classe, faz login com `ana@x.com` sem cadastrá-lo. Que letra do F.I.R.S.T. é violada e como a classe base do ServiceHub evita o problema?
8. Cite três motivos concretos para o ServiceHub não usar H2 nos testes de integração.
9. No teste de concorrência, por que a asserção é `containsExactlyInAnyOrder(OK, CONFLICT)` e não `containsExactly(OK, CONFLICT)`? Para que serve a `CyclicBarrier` e o `@RepeatedTest(5)`?
10. Um colega comemora 100% de cobertura de linhas no `ReviewService`. Dê dois motivos pelos quais a RN09 ainda pode estar mal testada.
11. Escreva um prompt, no formato desta semana, para construir em TDD a RN10 (usuário ou serviço com ordens ativas não pode ser excluído: 409). Liste os nomes dos testes.

15. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- GONÇALVES, P. F. et al. *Testes de software e gerência de configuração*. Grupo A, 2019. Capítulos "Níveis de teste", "Testes unitários" e "Teste de software com JUnit".
- MARTIN, R. C. *Código limpo*. Alta Books, 2009. Capítulo 9, testes de unidade (as três leis do TDD e F.I.R.S.T.).
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*. 9. ed. Grupo A, 2021. Capítulos 19 e 20.
- MARTIN, R. C. *Arquitetura limpa*. Alta Books, 2019. Capítulo 28, "O Limite Teste".
- MARTIN, R. C. *Desenvolvimento ágil limpo*. Alta Books, 2020. Capítulo 5, práticas técnicas e o ciclo do TDD.

Referências abertas:

- JUnit 5 User Guide: `junit.org/junit5/docs/current/user-guide`.
- Spring Boot Reference, seções *Testing* e *Testcontainers*: `docs.spring.io/spring-boot`.
- Testcontainers for Java, módulo PostgreSQL: `java.testcontainers.org`.
- Documentação do JaCoCo: `www.jacoco.org/jacoco`.